

ALGORITHM

Spring 2025, Midterm Exam, April 9

5:00 – 8:00 pm

1. (15%) Illustrate the operation of the following sorting algorithms respectively on the array $A = \langle 3, 7, 1, 4, 0, 6, 4, 2, 1, 3, 4 \rangle$, where $A[j] \in \{0, 1, \dots, 7\}$ for $1 \leq j \leq 11$.

Which of them are stable sorting algorithms? Which of them are in-place sorting algorithms?

(a) MERGE SORT

(b) QUICKSORT

(c) HEAPSORT

2. (20%) Let $f(n)$ and $g(n)$ be real-valued functions. The asymptotic notations, O , Ω , and Θ , are defined as follows.

- $f(n)$ is *asymptotically upper bounded by $g(n)$* , denoted by $f(n) = O(g(n))$,
iff there exists positive constants c and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n > n_0$.
- $f(n)$ is *asymptotically lower bounded by $g(n)$* , denoted by $f(n) = \Omega(g(n))$,
iff there exists positive constants c and n_0 such that $f(n) \geq c \cdot g(n)$ for all $n > n_0$.
- $f(n)$ is *asymptotically equivalent to $g(n)$* , denoted by $f(n) = \Theta(g(n))$,
iff there exists positive constants c_1 , c_2 and n_0 such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$
whenever $n > n_0$.

Please answer the following questions according to the definitions given above.

- (a) Finding upper bound for the function $f(n) = n^2 + 2n - 6$ by using the O notation.
Justify your answer by demonstrating the constants.
- (b) Finding lower bound for the function $f(n) = n^2 + 2n - 6$ by using the Ω notation.
Justify your answer by demonstrating the constants.
- (c) Finding tight bound for the function $f(n) = n^2 + 2n - 6$ by using the Θ notation.
Justify your answer by demonstrating the constants.
- (d) Show that $n^2 + 2n - 6 \neq \Theta(n^4)$.

3. (15%) Use a recursion tree to determine a good asymptotic upper bound on the recurrence $T(n) = 3T(\lfloor n/3 \rfloor) + 2n$, where $T(1) = 2$ and $T(2) = 6$. Use the substitution method to verify your answer.

4. (15%) Given an array $A = \langle x_1, x_2, \dots, x_n \rangle$ of numbers, the maximum-subarray problem is to find two indices i and j such that $A = \langle x_i, x_2, \dots, x_j \rangle$ has the greatest sum of any contiguous subarray of array A .
- Illustrate the operation of FIND-MAXIMUM-SUBARRAY on the array $A = \langle -3, 7, -2, 4, -3, 3, 4, -7 \rangle$.
 - Write pseudocode for the brute-force method of solving the maximum-subarray problem. Your procedure should run $\Theta(n^2)$ time.
5. (15%) Please answer the following questions.
- Use the master method to give a tight asymptotic bound on the running time of the recurrence $T(n) = 7T(n/2) + n^3$.
 - Illustrate the operation of MAX-HEAP-INSERT($A, 3$) on the heap $A = \langle 15, 13, 9, 10, 12, 8, 7, 4, 5, 6, 2, 1 \rangle$. node i from heap A . Give an implementation of HEAP-DELETE that runs in $O(\lg n)$ time for an n -element max-heap.
 - Give an example to show that the worst-case running time of MAX-HEAPIFY on a heap of size n is $\Omega(\lg n)$.
6. (15%) A lazy programmer thinks that implementing the procedure MERGE (of the merge sort) is too complicated. To simplify MERGE-SORT he eliminates the merge all together, and only recursively “sorts” the two halves of the array. Here is his pseudo code.

LAZY-SORT(A, p, r)

```

1  if  $p < r$ 
2       $q \leftarrow \lfloor (p + r)/2 \rfloor$ 
3      LAZY-SORT( $A, p, q$ )
4      LAZY-SORT( $A, q + 1, r$ )

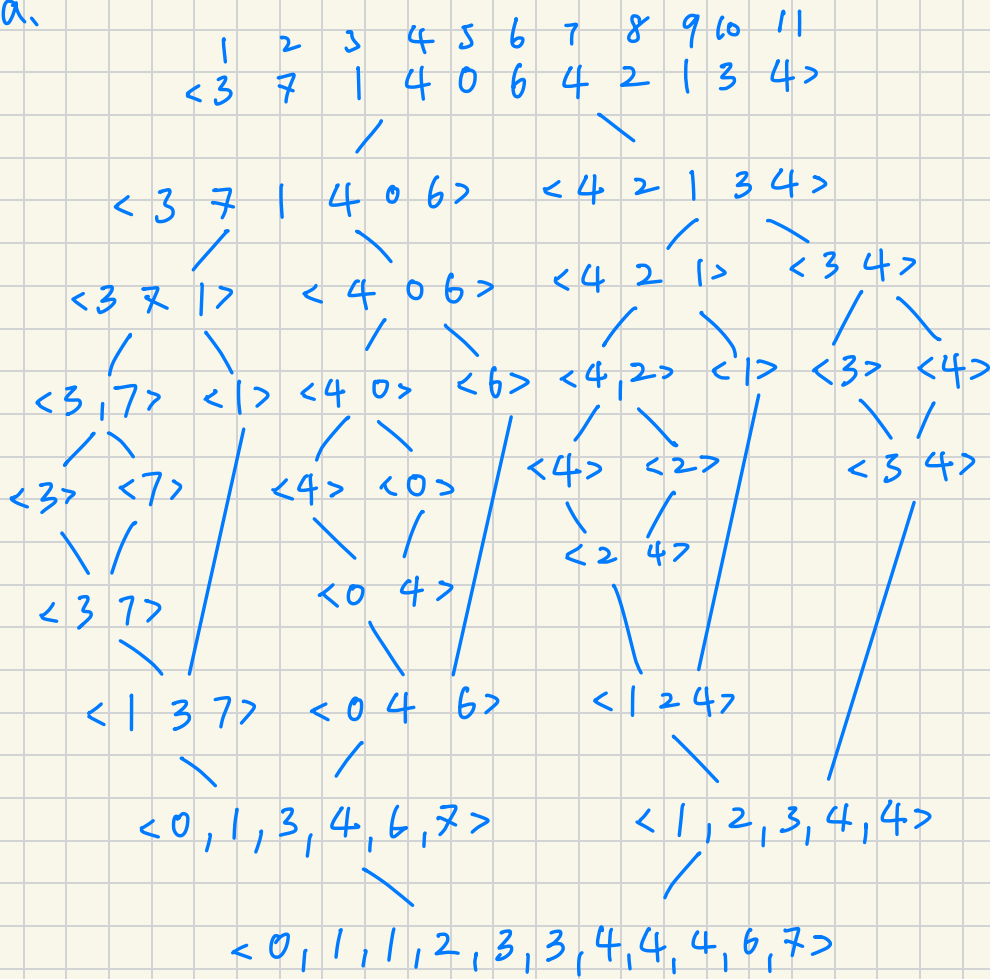
```

Initial call: LAZY-SORT($A, 1, n$)

- What does LAZY-SORT do?.
 - What is the running time of LAZY-SORT?
7. (15%) Finding a missing number. An array of n elements contains all but one of the integers from 1 to $n + 1$.
- Give the best algorithm you can for determining which number is missing if the array is sorted, and analyze its asymptotic worst-case running time.
 - Give the best algorithm you can for determining which number is missing if the array is “not” sorted, and analyze its asymptotic worst-case running time.

1, <3, 7, 1, 4, 0, 6, 4, 2, 1, 3, 4>

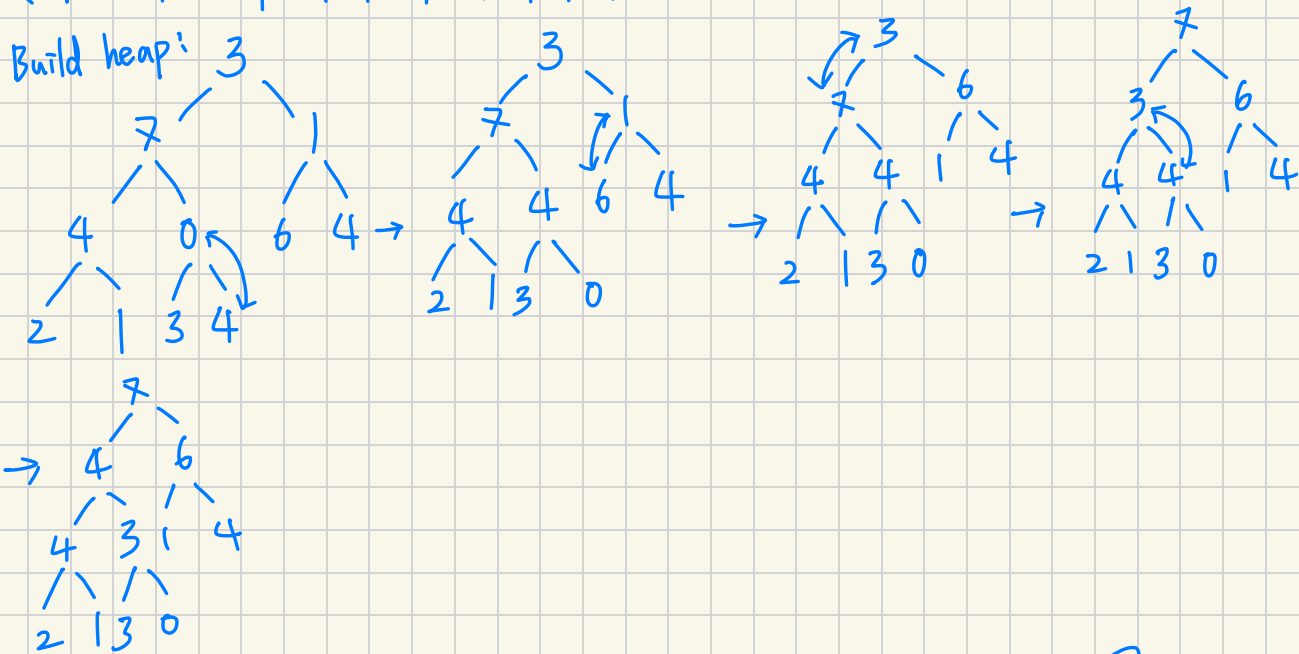
a.



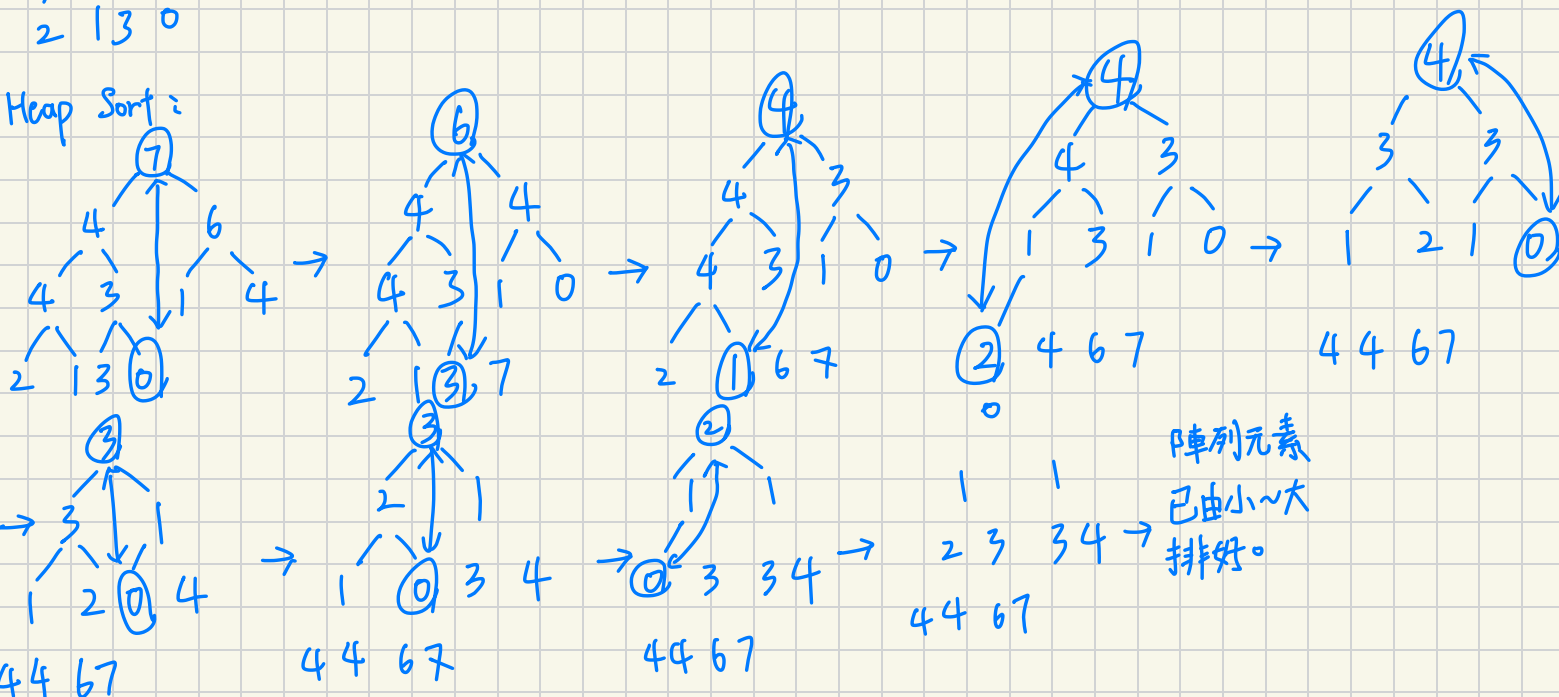
[illegible]

(c) $\langle 3, 7, 1, 4, 0, 6, 4, 2, 1, 3, 4 \rangle$ (7 ~ 1)

Build heap:



Heap Sort:



Stable : merge sort

In-Place : quick sort, heap sort

2、

(a) let $f(n) = n^2 + 2n - 6$, $g(n) = n^2$

$f(n) = O(g(n))$ if there exist positive constants C, n_0 such that $0 \leq f(n) \leq Cg(n)$

$\forall n \geq n_0$

$$n^2 + 2n - 6 \leq Cn^2 \xrightarrow{\text{同除 } n^2} 1 + \frac{2}{n} - \frac{6}{n^2} \leq C$$

隨 n 增長, 當 $n \rightarrow \infty$ 時, $1 + \frac{2}{n} - \frac{6}{n^2}$ 會趨近於 1

s.t. $1 \leq C$

$\rightarrow C$ 取 2, n_0 取 2

當 $C=2, n_0=2$ 時, 存在 $n^2 + 2n - 6 \leq 2n^2, \forall n \geq 2$

$\therefore$ 可證, $n^2 + 2n - 6 = O(n^2)$

$$n=1, 1+2-6=-3 \leq C$$

$$n=2, 1+1-\frac{6}{4}=\frac{1}{2} \leq C$$

$$n=3, 1+\frac{2}{3}-\frac{6}{9}=1 \leq C$$

$$n=4, 1+\frac{2}{4}-\frac{6}{16}=\frac{18}{16} \leq C$$

$$n=5, 1+\frac{2}{5}-\frac{6}{25}=\frac{29}{25} \leq C$$

(b) let $f(n) = n^2 + 2n - 6$, $g(n) = n^2$

$f(n) = \Omega(g(n))$ if there exist positive constants C, n_0 such that $0 \leq Cg(n) \leq f(n) \quad \forall n \geq n_0$

$$Cn^2 \leq n^2 + 2n - 6 \xrightarrow{\text{同除 } n^2} C \leq 1 + \frac{2}{n} - \frac{6}{n^2}$$

$n=1 \quad C \leq 1 + 2 - 6 = -3$

$n=2 \quad C \leq 1 + 1 - \frac{6}{4} = \frac{1}{2}$

$n=3 \quad C \leq 1 + \frac{2}{3} - \frac{6}{9} = 1$

$n=4 \quad C \leq 1 + \frac{2}{4} - \frac{6}{16} = \frac{18}{16} \left(1 \frac{1}{4}\right)$

$n=5 \quad C \leq 1 + \frac{2}{5} - \frac{6}{25} = \frac{29}{25} \left(1 \frac{4}{25}\right)$

$\vdots$

隨 n 增長, 當 $n \rightarrow \infty$ 時, $1 + \frac{2}{n} - \frac{6}{n^2}$ 會趨近於 1,
s.t. $C \leq 1$

C 取 $\frac{1}{2}$, n_0 取 2

當 $C = \frac{1}{2}$, $n_0 = 2$ 時, 存在 $\frac{1}{2}n^2 \leq n^2 + 2n - 6 \quad \forall n \geq 2$

$\therefore$ 可證 $n^2 + 2n - 6 = \Omega(n^2)$

(c) let $f(n) = n^2 + 2n - 6$, $g(n) = n^2$

$f(n) = \Theta(g(n))$ if there exist positive constants C_1, C_2 and n_0 such that $C_1g(n) \leq f(n) \leq C_2g(n)$

$\forall n \geq n_0$

$$C_1n^2 \leq n^2 + 2n - 6 \leq C_2n^2 \xrightarrow{\text{同除 } n^2} C_1 \leq 1 + \frac{2}{n} - \frac{6}{n^2} \leq C_2$$

$$C_1 \leq 1 + \frac{2}{n} - \frac{6}{n^2} \quad 1 + \frac{2}{n} - \frac{6}{n^2} \leq C_2$$

$n=1 \quad C_1 \leq -3$

$-3 \leq C_2$

$n=2 \quad C_1 \leq \frac{1}{2}$

$\frac{1}{2} \leq C_2$

$n=3 \quad C_1 \leq 1$

$1 \leq C_2$

$n=4 \quad C_1 \leq \frac{18}{16}$

$\frac{18}{16} \leq C_2$

$n=5 \quad C_1 \leq \frac{29}{25}$

$\frac{29}{25} \leq C_2$

$\vdots$

C_1 取 $\frac{1}{2}$

C_2 取 2

隨 n 增長, 當 $n \rightarrow \infty$ 時, $1 + \frac{2}{n} - \frac{6}{n^2}$ 會趨近於 1,
s.t. $C_1 \leq 1 \leq C_2$

$\rightarrow C_1$ 取 $\frac{1}{2}$, C_2 取 2, n_0 取 2

當 $C_1 = \frac{1}{2}$, $C_2 = 2$, $n_0 = 2$ 時, 存在

$\frac{1}{2}n^2 \leq n^2 + 2n - 6 \leq 2n^2, \quad \forall n \geq n_0$

$\therefore$ 可證, $n^2 + 2n - 6 = \Theta(n^2)$

(d) show $n^2 + 2n - 6 \neq \Theta(n^4)$

if $n^2 + 2n - 6 = \Theta(n^4)$, then exist positive C_1, n_0 such that $C_1g(n) \leq f(n) \quad \forall n \geq n_0$

$$C_1n^4 \leq n^2 + 2n - 6 \xrightarrow{\text{同除 } n^2} C_1n^2 \leq 1 + \frac{2}{n} - \frac{6}{n^2}$$

其中, 隨著 n 的增長, 右式 $1 + \frac{2}{n} - \frac{6}{n^2}$ 會漸趨近於 1, 但左式 C_1n^2 卻會隨 n 指數級放大, 必不可能 ≤ 1 , 這與假設 $\forall n \geq n_0$ 矛盾。

$\therefore$ 可證, $n^2 + 2n - 6 \neq \Theta(n^4)$

$$3. T(n) = 3T(\lfloor \frac{n}{3} \rfloor) + 2n, T(1) = 2, T(2) = 6$$

層數 每層花費

第k層 total cost:

$$(\frac{2n}{3^k}) \times 3^k = 2n$$

let recursion tree total h 層

層數	0	1	2		h-1
子問題大小	2n	$\frac{n}{3}$	$\frac{n}{9}$		$\frac{n}{3^{h-1}}$

$$\frac{n}{3^{h-1}} = 1, n = 3^{h-1}, h = \log_3 n + 1$$

$$T(n) = 2n + 2n + 2n + \dots + 2n = 2 \sum_{i=0}^{\log_3 n} n = 2 \times [n \times (\log_3 n + 1)] = 2n \log_3 n + 2n \Rightarrow T(n) = O(n \lg n)$$

substitution method verify $\rightarrow$ big O: $C \cdot n_0$

Basic Step:

$$T(1) = 2 \leq C_1 \lg 1 = C_1 \times 0 \Rightarrow C_1 \text{ 不存在}$$

$$T(2) = 6 \leq 2C_1 \lg 2 \Rightarrow C_1 \geq 5$$

$$T(3) = 3T(1) + 6 = 12 \leq 3C_1 \lg 3 = 3C_1 \Rightarrow C_1 \geq 4$$

$$\left. \begin{array}{l} C_1 = 5, n_0 = 2 \end{array} \right\}$$

Induction Step:

$$\text{Assume: } T(\lfloor \frac{n}{3} \rfloor) \leq C_2 \lfloor \frac{n}{3} \rfloor \lg(\lfloor \frac{n}{3} \rfloor) \text{ for } \lfloor \frac{n}{3} \rfloor$$

$$\text{then: } T(n) = 3T(\lfloor \frac{n}{3} \rfloor) + 2n \leq 3(C_2 \lfloor \frac{n}{3} \rfloor \lg(\lfloor \frac{n}{3} \rfloor)) + 2n$$

$$\leq C_2 n \lg \lfloor \frac{n}{3} \rfloor + 2n$$

$$\leq C_2 n \lg n - C_2 n \lg 3 + 2n$$

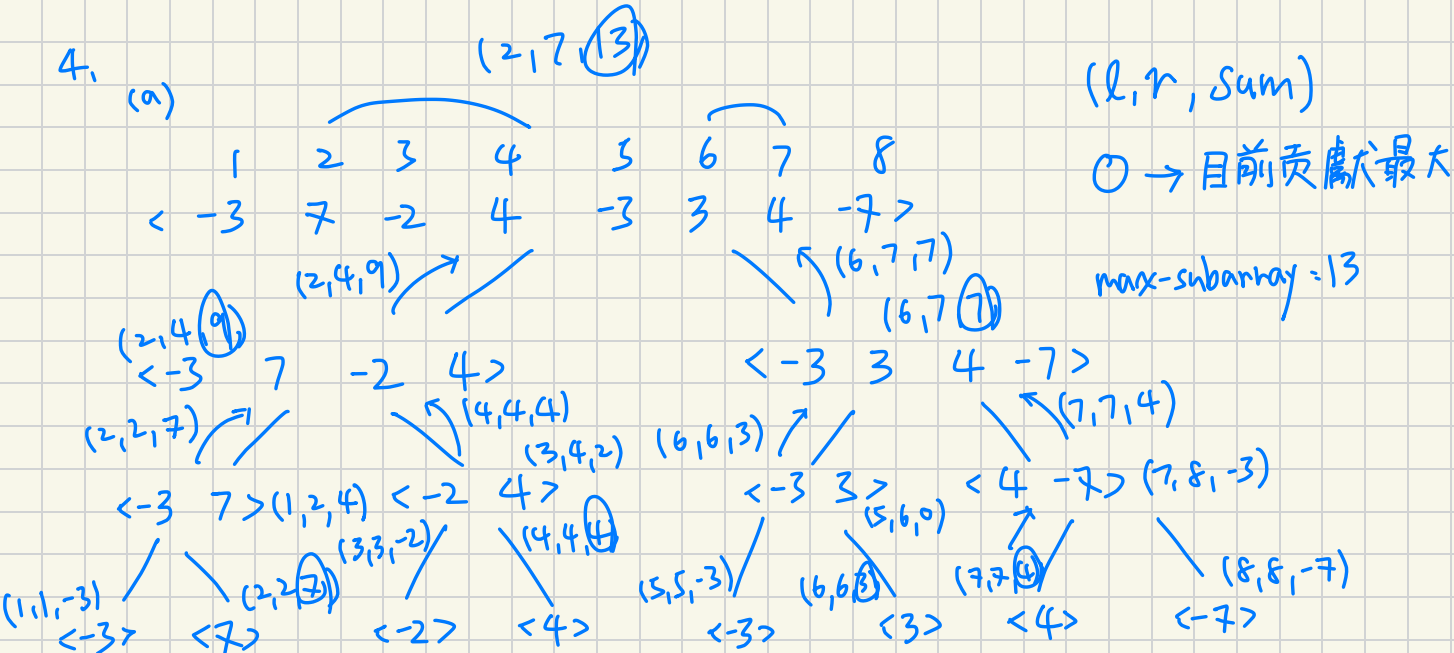
$$= C_2 n \lg n - C_2 n + 2n$$

$$\leq C_2 n \lg n \rightarrow \text{當 } C_2 \geq 1 \text{ 時, 恆成立。}$$

$$\therefore \exists C = \max(C_1, C_2) = 5, n_0 = 2, \text{ s.t. } T(n) \leq C n \lg n \quad \forall n \geq n_0$$

4.

(a)



(b)

Find-maximum-subarray-brute (A):

maxSum $\leftarrow -\infty$ l $\leftarrow 1$ r $\leftarrow 1$ for $i = 1 \sim \text{length}(A)$ nowSum $\leftarrow 0$ for $j = i \sim \text{length}(A)$

nowSum = nowSum + A[j]

if nowSum > maxSum:

maxSum $\leftarrow$ nowSuml $\leftarrow i$ r $\leftarrow j$

return (l, r, maxSum)

5.

$$(a) T(n) = 7T\left(\frac{n}{2}\right) + n^3$$

$$f(n) = n^3, \log_b a = \log_2 7 \approx 2.8$$

$$n^{2.8} < n^3, 3 - 2.8 = 0.2 > 0$$

Regularity Cond.

$$7f\left(\frac{n}{2}\right) \leq Cn^3$$

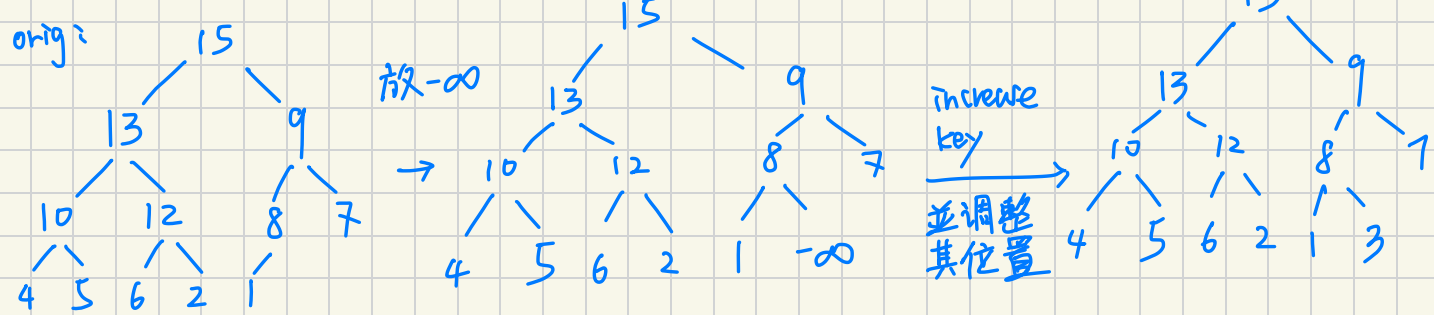
$$7 \times \frac{n^3}{8} \leq Cn^3$$

$$\frac{7}{8}n^3 \leq Cn^3, C \geq \frac{7}{8} \leq 1$$

$\therefore$ by Case 3, $T(n) = \Theta(n^3)$

(b)

insert:



delete:

Heap-Delete (A, i)last $\leftarrow A[A.\text{heapsize}]$ $A.\text{heapsize} \leftarrow A.\text{heapsize} - 1$ if $i \leq A[A.\text{heapsize}]$ if last $> A[\text{parent}(i)]$ increasekey(A, i, last)

else

 $A[i] = \text{last}$ max-heapify(A, i)

(c)

consider a max heap, $h = \lfloor \lg n \rfloor$ $A[1] = 0$ (root), 其所有子孫節點都 > 0 。當執行 max-heapify($A, 1$) 時, 該 node 需和其 child 比較、交換。由於 $A[1]$ 是最小的, 所以每次都會被壓中並交換, 直到 leaf 為止。總交換次數: $\lg n$, 交換比較: $(1) \rightarrow \Omega(\lg n)$

7.

(a) sorted: binary search $O(\lg n)$ (b) randomized: 亂數 / XOR $O(n)$

